



Free Fingerprint Verification SDK 3.1

Developer's Guide

Table of Contents

Preface	1
Copyright Notice	1
Questions	1
Feedback	1
Introduction	2
About This Guide	2
How the Guide Is Organized	2
Target Audience	2
Additional Resources	2
About Neurotechnology	3
Free SDK vs. VeriFinger SDK	3
Online Resources	3
System Requirements	4
Fingerprint Scanners	4
Quick Start	5
Terminology	5
Fingerprints	5
Enrollment	5
Verification	6
Quality Threshold	6
Matching Threshold	6
How to Use Fingerprint Scanner	7
Using Sample Applications	7
API Reference	12
C/C++ Reference	12
Nffv Library	12
Nffv Unit	12
Functions	13
Structs, Records, Enums	25
Macros	26

.NET Reference	26
Neurotec.Biometrics Namespace	27
Classes	27
Nffv Class	27
NffvUser Class	34
Structs, Records, Enums	35
Neurotec.Biometrics.NffvStatus Enumeration	35
Java Reference	36
com.neurotechnology.Nffv Package	36
Classes	36
Nffv Class	37
NffvImage Class	41
NffvUser Class	44
ScannerModule Class	45
Structs, Records, Enums	46
com.neurotechnology.Nffv.NffvStatus Enumeration	46
Distribution Content	48
Error Codes	49
FAQ	51
Index	a

Free Fingerprint Verification SDK 3.1

1 Preface

The brief information about this developer guide.

Version: 3.1.0.0

Release date: 3/30/2026

1.1 Copyright Notice

The Software is Copyright (c) 2008-2026 Neurotechnology. All rights reserved. The Software remains the sole and exclusive property of Neurotechnology at all times.

1.2 Questions

After you have read this developer's guide and the FAQ (see page 51) chapter and still have more questions or face troubleshoots in using the Free Fingerprint Verification SDK, please feel free to contact us.

Contacts

- **Email:** support@neurotechnology.com. When writing by email thoroughly describe a problem you face. Also you can attach additional files (eg. screenshots, fingerprint images) which can help solve your problem.

1.3 Feedback

The information in this guide has been thoroughly reviewed and tested, but if you have noticed errors, omissions or have suggestions for future improvements, please send us feedback via email support@neurotechnology.com.

2 Introduction

Free Fingerprint Verification SDK allows to develop biometric applications that verify a scanned fingerprint against a previously stored fingerprint (1-to-1 matching). There is a simple migration path to VeriFinger SDK which is a commercial SDK that offers state-of-the art proprietary accuracy and speed with more capabilities for software developers.

Free Fingerprint Verification SDK does not allow to develop applications that read fingerprint images from files or perform fingerprint identification (1-to-many matching). The number of stored fingerprint templates is limited to 10 records in the application's database.

The VeriFinger SDK allows to develop a wide range of biometric applications that identify fingerprints taken from fingerprint scanner or image files against fingerprints stored in a database (1-to-many matching). Also, the VeriFinger SDK does not have any limitations on fingerprint database size and allows to develop both standalone PC-based and Web-based application that use client-server architecture.

The VeriFinger SDK contains a wrapper that allows easy transition from Free Fingerprint Verification SDK to VeriFinger SDK.

2.1 About This Guide

This document is a developer guide on writing biometrical applications with FFV SDK. When developing your own applications you must be proficient in at least one of these programming languages: C++, C, C#, Java, VB .NET. Also a basic knowledge of biometrical systems is desirable.

2.1.1 How the Guide Is Organized

Chapter *Introduction* (see page 2) focuses on the general information about FFV SDK.

Chapter *Quick Start* (see page 5) provides a quick introduction to the FFV SDK and discusses how to use a fingerprint scanner and sample applications. Also fingerprints enrollment and verification operations are explained.

The components for developing applications that uses the functionality of the FFV SDK are defined in chapter *API Reference* (see page 12).

Answers to frequently asked questions are reviewed in chapter *FAQ* (see page 51).

2.1.2 Target Audience

This guide is for developers who have a working experience in programming with at least one of these programming languages: C, C++, C#, Java or VB .NET.

2.2 Additional Resources

This chapter provides additional resources that can help you using the FFV SDK.

2.2.1 About Neurotechnology

Neurotechnology was founded in Vilnius, Lithuania in 1990 with the key idea of using neural networks for applications such as biometric person identification, computer vision, robotics and artificial intelligence. Much to our delight, we were able to endure the "neural networks winter" by using and expanding this expertise all through 2012, the year that brought explosive developments in the concept and infrastructure of deep neural networks. This allowed us to quickly take advantage of the emerging opportunities that came with the new wave of deep learning and triggered an entire range of new projects in object recognition and other applications. Currently, our team is comprised of more than 100 employees, 15% of whom hold a Ph.D. and half of our employees are actively involved in R&D activities.

See Also

Read more about [Neurotechnology](#).

2.2.2 Free SDK vs. VeriFinger SDK

VeriFinger SDK is intended for biometric system developers and integrators and allows a rapid development of large-scale biometric applications based on fingerprint verification. It also offers state-of-the-art proprietary accuracy and speed. Also the VeriFinger SDK can have an additional component - Matching server which performs the identification and verification of fingerprints on server side. There is a simple migration path from Free Fingerprint Verification SDK to VeriFinger SDK.

Differences between Free Fingerprint Verification SDK and Verifinger SDK are listed in the table below:

Feature	FFV SDK	VeriFinger SDK
Fingerprint scanners support	+	+
Fingerprint verification against live scanned image	+	+
High speed identification against database		+
Fingerprint features template extraction from image		+
Programming samples and tutorials	+	+
State-of-the-art proprietary accuracy and speed		+
Database template count*	10	unlimited
Support for Windows operating systems (7 / 8 / 10)	+	+
Support for Linux operating systems		+
Support for macOS operating system		+
Support for 64 bit operating systems (Windows and Linux)		+

* Database template count is a maximum number of fingerprints that can be saved to a database.

If you need more information about VeriFinger SDK, please visit https://www.neurotechnology.com/vf_sdk.html.

2.2.3 Online Resources

If you need more information about the company or products you can refer to one of these online resources:

Link	Description
https://www.neurotechnology.com/	The Neurotechnology home page. Provides the latest news and updates of Neurotechnology products.
https://download.neurotechnology.com/Scanner_Drivers.zip	Fingerprint scanner drivers.

2.3 System Requirements

The minimal hardware and software requirements needed to run Neurotechnology Free Fingerprint Verification SDK are listed:

- Computer with x86 compatible processor running at 2GHz or better.
- 2 GB of free RAM is recommended.
- Microsoft Windows 7/8/10 operating system.
- Microsoft .NET framework 4.5 (for .NET components);
- Microsoft Visual Studio .Net 2012 or newer.
- Java SE JDK 8 or newer (for application development with Java).
- A fingerprint scanner (see page 4) that is supported by Free Fingerprint Verification SDK.

2.4 Fingerprint Scanners

The list of supported fingerprint scanners is provided at Neurotechnology's [website](#).

3 Quick Start

This chapter provides a quick introduction to Free Fingerprint Verification SDK. In this chapter is discussed:

- The basic terminology related to fingerprints.
- The instructions for using a fingerprint scanner.
- A guide for using sample applications which are included in Free Fingerprint Verification SDK.

3.1 Terminology

The following two main definitions which are the cornerstone of biometric systems that use fingerprint identification will help you understand the Neurotechnology Free Fingerprint Verification SDK functionality:

- Enrollment (see page 5) is the process of capturing a person's fingerprint (using a fingerprint capture device). During the enrollment process the FFV SDK saves a person's fingerprint to a database.
- Verification (see page 6) is the process when a captured fingerprint is compared to an enrolled fingerprint in order to determine whether the two match.

Fingerprint enrollment and verification processes are described in detail in chapter Fingerprints (see page 5).

3.2 Fingerprints

A **fingerprint** is an impression of the friction ridges of all or any part of the finger. Fingerprint recognition systems use characteristics from these ridges (they are also called fingerprint features) to differentiate one fingerprint from another. The Free Fingerprint Verification SDK converts these features to a format (a template) that enables to perform fingerprint enrollment and verification operations efficiently and qualitatively.

3.2.1 Enrollment

Fingerprint enrollment is a process during which features from a finger are extracted and saved as a fingerprint template for a future comparison against other fingerprint templates. The following instructions describe a typical fingerprint enrollment scheme (the same scheme is used in sample applications):

1. Get a person's identification number.
2. Capture a person's fingerprint using a fingerprint scanner.
3. Extract a fingerprint features from a fingerprint image.
4. Associate a person with his fingerprint.
5. Save extracted features (a template) to a database.

To enroll a fingerprint you can use these functions:

```
//C/C++ function
NResult N_API NffvEnroll(NUInt timeout, NffvStatus * pStatus, HNffvUser * pHUser);
//.NET method
```

```
public NffvUser Enroll(uint timeout, out NffvStatus status);
```

3.2.2 Verification

Fingerprint verification is a process during which a scanned fingerprint is compared with the one saved to a database and is decided whether the two match. The following scheme is usually used for a fingerprint verification:

1. Get a person's identification number.
2. Capture a person's fingerprint using a fingerprint scanner.
3. Extract a fingerprint features from a fingerprint image for the purpose of verification.
4. Get a fingerprint template (the one that was saved to a database earlier) by identification number
5. Compare two fingerprints: the one that was scanned with the one that was saved to database.
6. Perform an action according to the verification result (eg. unlock a computer if two fingerprints matches).

To verify a fingerprint you can use these functions:

```
//C/C++ function
NResult N_API NffvVerify(HNffvUser hUser, NUInt timeout, NffvStatus * pStatus, NInt *
pScore);
//.NET method
public int Verify(NffvUser user, uint timeout, out NffvStatus status);
```

3.3 Quality Threshold

Quality threshold is the property that defines a scanned fingerprint image quality. Specifies the threshold which is considered when extracting fingerprint features from the image. With higher threshold better quality of fingerprint image is required to successfully extract fingerprint features. The Quality threshold should be in range [0, 255]. 255 means the best image quality.

The default quality threshold value is 100.

In the Free Fingerprint Verification SDK quality threshold can be set using these functions:

```
//.NET property
public byte QualityThreshold;
//C/C++ function
NResult N_API NffvSetQualityThreshold(NByte value);
```

3.4 Matching Threshold

Matching threshold - the minimum similarity value that verification and identification functions accept for the same finger fingerprints or face.

Matching threshold should be selected according to the system's development requirements. The default value is 48.

In the Free Fingerprint Verification SDK matching threshold can be set using these functions and properties:

```
//.NET property
public int MatchingThreshold;
//C/C++ function
NResult N_API NffvSetMatchingThreshold(NInt value);
```

3.5 How to Use Fingerprint Scanner

A **fingerprint scanner** is a device connected to computer and used for capturing a person's fingerprint image. Depending on scanner's manufacturer and model it can be connected to USB or Ethernet port. In order to use a fingerprint scanner with the Free Fingerprint Verification SDK you should do the following:

- Insert a fingerprint scanner into the USB or Ethernet connector on the system where you copied the Free Fingerprint Verification SDK files.
- Install the scanner drivers whether the one you got from a manufacturer (yours) or download from Neurotechnology's [website](#).

3.6 Using Sample Applications

The Free Fingerprint Verification SDK contains sample applications which demonstrates the functionality of the SDK. You are free to adjust these applications for your needs.

The sample applications are located in "*Samples*" folder. If you want to test the sample application from this folder, you must first compile or build files from this folder.

There are samples in the following programming languages:

- C++ (*\Samples\FreeFingerprintVerification\Cpp*)
- C# (*\Samples\FreeFingerprintVerification\CS*)
- VB.NET (*\Samples\FreeFingerprintVerification\VB*)
- Java (*\Samples\FreeFingerprintVerification\Java*)

This chapter explains in detail how to use C# sample applications and comments the source code. The usage of samples in different programming languages are very similar.

By reading this section you will

- Open a sample application project file and build it
- Enroll a fingerprint
- Make a verification of a fingerprint

If you want to test a sample application without building it, you can find an executable file in *\Bin\Win64_x64*.

Using C# sample application

1. Starting the sample application

Open the solution file using Microsoft Visual Studio located in the folder "*\Samples\CS\CSharpSample\CSharpSample.sln*".

The C# sample solution project contains these main files:

- *AboutForm.cs*. This file is used for showing a basic information about a sample application.
- *ChooseScannerForm.cs*. This file is used for showing a dialog box for selecting a fingerprint scanner.
- *SettingsForm.cs*. This file is used for showing a form where matching and quality thresholds can be set.
- *MainForm.cs*. This file contains all the main functionality of the application (also methods for fingerprint enrollment and verification).
- *UserInfoForm.cs*. This file contains properties that enable to get or set a user name and fingerprint.

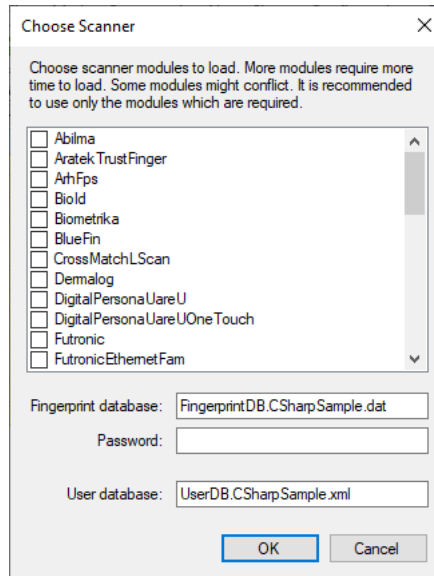
Also you should notice that the solution project contains references to this dynamic-link library (DLL):

- **Neurotec.Biometrics.Nffv.dll.** This library is the main DLL for your solution projects and provides the enrollment and verification of a fingerprint functionality.

This library can be located in the "*/Bin/Win64_x64*" folder of the FFV SDK.

2. Selecting a fingerprint scanner

When you have built the sample application solution project and launched it, the dialog box for selecting a scanner appears:



There are listed scanner models supported by the FFV SDK. Select only the scanner models you will use. You should note that more modules require more time to load.

Enrolled fingerprints will be saved to a database (see a fingerprint database field). You can protect this database by setting a password. Person's details are saved to users database (in this sample application users database is an Xml file) where a person's name and his fingerprint ID is saved. You can implement your own users database by adding more fields.

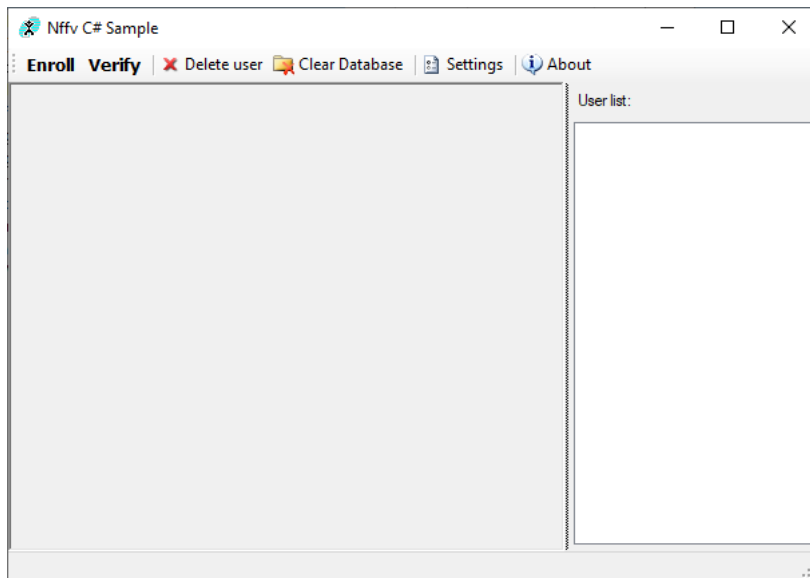
To get the list of scanners supported by the FFV SDK, you can use the following code:

```
//A string variable which contains a list of scanner modules
private string scannerModules = string.Empty;

//Gets a list of supported scanners
scannerModules = NFEngine.GetAvailableScannerModules();
```

3. The main window of the sample application

After you have selected fingerprint scanners, created databases and pressed the OK button, the main window of the sample application appears:



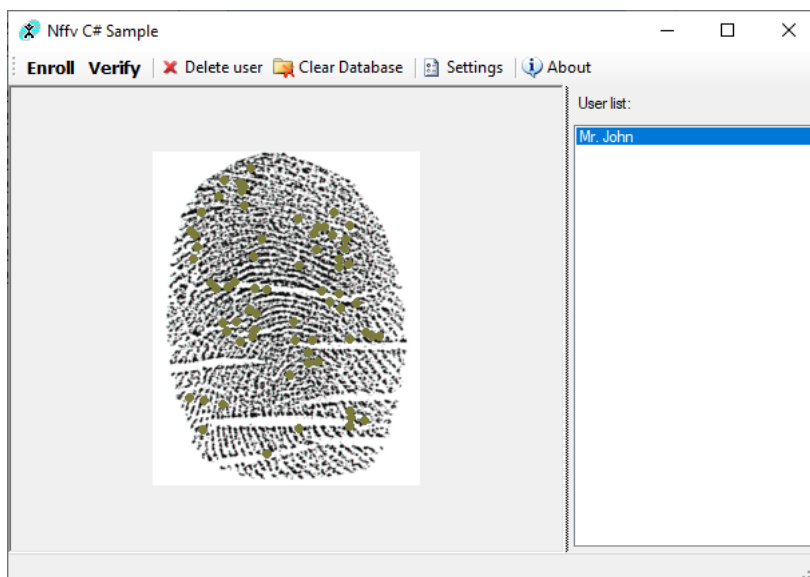
Below are listed operation you can do using this sample application:

- Enroll
- Verify
- Delete user
- Clear database
- Settings
- About

Now let's discuss these operations in detail and illustrate them using C# source code.

4. Enrolling a fingerprint

With the purpose of enrolling a fingerprint to database a fingerprint scanner should be connected to a computer. The fingerprint is enrolled by pressing "Enroll" (the dialog box shows up and asks for a name of a person). After the enrollment of a person's fingerprint has finished you should see a window like this:



A fingerprint image and the name of a person (let's say it is Mr. John) is shown on a window.

Using The FFV SDK you can enroll up to 10 records to a database.

Person's fingerprint can be enrolled using this C# method:

```

public void EnrollUser ( )
{
    RunWorkerCompletedEventArgs taskResult = BusyForm.RunLongTask("Waiting for fingerprint
    ...",
        new DoWorkEventHandler(doEnroll),
        false, null, new EventHandler(CancelScanningHandler));
    EnrollmentResult enrollmentResult = (EnrollmentResult)taskResult.Result;
    if (enrollmentResult.engineStatus == NffvStatus.TemplateCreated)
    {
        NffvUser engineUser = enrollmentResult.engineUser;
        string userName = enrollDlg.UserName;
        if (userName.Length <= 0)
        {
            userName = engineUser.Id.ToString();
        }

        _userDB.Add(new UserRecord(engineUser.Id, userName));
        try
        {
            _userDB.WriteToFile(_userDatabaseFile);
        }
        catch { }

        pbExtractedImage.Image = engineUser.GetBitmap();
        lbDatabase.Items.Add(new ListBoxImage.CData(engineUser, userName));
        lbDatabase.SelectedIndex = lbDatabase.Items.Count - 1;
    }
    else
    {
        NffvStatus engineStatus = enrollmentResult.engineStatus;
        MessageBox.Show(string.Format("Enrollment was not finished. Reason: {0}",
engineStatus));
    }
}

```

5. Verifying a fingerprint

When you need to verify a person's fingerprint with the one that was enrolled to a database you should select a database record and press the "Verify" button. After your fingerprint is scanned the verification is made. If the two fingerprints are identical, the matching score is shown. Otherwise, a message box announcing that fingerprints are not identical is shown.

Person's scanned fingerprint verification can be made using this C# method:

```

private void doVerify(object sender, DoWorkEventArgs args)
{
    VerificationResult verificationResult = new VerificationResult();
    verificationResult.score = _engine.Verify((NffvUser)args.Argument, 20000,
        out verificationResult.engineStatus);
    args.Result = verificationResult;
}

```

6. Deleting a user

To delete a user from a database you can use this C# method:

```

private Neurotec.Biometrics.Gui.ListBoxImage lbDatabase;
UserDatabase _userDB;
Nffv _engine;

public void DeleteUser ( )
{
    if (lbDatabase.SelectedIndex < 0)
    {
        MessageBox.Show("Please select a record from the database.");
    }
    else
    {
        _userDB.Remove(_userDB.Lookup(((ListBoxImage.CData)lbDatabase.SelectedItem).ID));
        try
        {
            _userDB.WriteToFile(_userDatabaseFile);
        }
    }
}

```

```

        catch { }

        _engine.Users.RemoveAt(lbDatabase.SelectedIndex);
        lbDatabase.Items.RemoveAt(lbDatabase.SelectedIndex);
    }
}

```

7. Clearing a database

When you need to delete all records from a database you can use this C# method:

```

//Fields
Nffv _engine;
private Neurotec.Biometrics.Gui.ListBoxImage lbDatabase;
UserDatabase _userDB;

public void ClearDatabase ( )
{
    if (MessageBox.Show("All records will be deleted from database. Do you want to
continue?",
                        "Confirm delete", MessageBoxButtons.YesNo,
                        MessageBoxIcon.Question) != DialogResult.Yes)
    {
        return;
    }

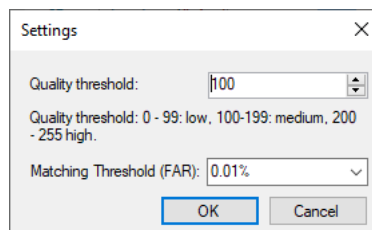
    //Clears a database
    _engine.Users.Clear();
    lbDatabase.Items.Clear();

    _userDB.Clear();
    try
    {
        _userDB.WriteToFile(_userDatabaseFile);
    }
    catch { }
}

```

8. Settings

When you press the "Settings" button you will see a window like this:



For more information on setting quality and matching thresholds see chapters [Quality Threshold](#) (see page 6) and [Matching Threshold](#) (see page 6).

4 API Reference

This chapter defines the components for developing applications that uses the functionality of the Free Fingerprint Verification SDK.

Modules

Name	Description
.NET Reference (see page 26)	... more (see page 26)
Java Reference (see page 36)	... more (see page 36)

4.1 C/C++ Reference

This chapter provides the Free Fingerprint Verification SDK programming reference for C/C++ programming languages.

Remarks

If you are developing your own application using C or C++, you should link Nffv.dll.lib library to your solution project. Also Nffv.dll library is needed.

In order to use Nffv.dll the folder must contain NffvServer.exe file.

4.1.1 Nffv Library

Modules

Name	Description
Nffv Unit (see page 12)	

4.1.1.1 Nffv Unit

Functions

	Name	Description
≡	NffvCancel (see page 14)	Cancels a fingerprint enrollment or verification operation.
≡	NffvClearUsers (see page 15)	Removes all the users which were enrolled to a database.
≡	NffvEnroll (see page 15)	Gets a fingerprint from a scanner and saves it to a database.
≡	NffvFreeMemory (see page 16)	Releases memory allocated by the NffvGetAvailableScannerModules function..
≡	NffvGetAvailableScannerModulesA (see page 16)	Returns available fingerprint scanner modules for usage in the Free Fingerprint Verification SDK.
≡	NffvGetAvailableScannerModulesW (see page 16)	Returns available fingerprint scanner modules for usage in the Free Fingerprint Verification SDK.
≡	NffvGetErrorMessageA (see page 17)	Gets an error message. Use this function for errors handling.
≡	NffvGetErrorMessageW (see page 17)	Gets an error message. Use this function for errors handling.
≡	NffvGetInfoA (see page 17)	Retrieves information about the Nffv library.

	NffvGetInfoW (see page 18)	Retrieves information about the library.
	NffvGetMatchingThreshold (see page 18)	Gets the minimum similarity value that verification function uses to determine whether the fingerprint matches.
	NffvGetQualityThreshold (see page 19)	Gets an image quality threshold.
	NffvGetUser (see page 19)	Gets the information from a users list about an enrolled user.
	NffvGetUserById (see page 19)	Returns user details by the Id from a database.
	NffvGetUserCount (see page 20)	Retrieves the number of users enrolled to database.
	NffvGetUserIndexById (see page 20)	Retrieves the index from users list of a user indicated by the Id.
	NffvInitializeA (see page 21)	... more (see page 21)
	NffvInitializeW (see page 21)	... more (see page 21)
	NffvRemoveUser (see page 22)	Removes a user from users list (database).
	NffvSetMatchingThreshold (see page 22)	Sets the minimum similarity value that verification function uses to determine whether the fingerprint matches.
	NffvSetQualityThreshold (see page 23)	Sets an image quality threshold.
	NffvUninitialize (see page 23)	Releases memory resources.
	NffvUserGetHBitmap (see page 23)	Gets a handle to the bitmap of a user fingerprint.
	NffvUserGetId (see page 24)	Gets user Id by specified user details (NffvUser).
	NffvVerify (see page 24)	Compares a captured fingerprint with the one that was enrolled to a database before in order to determine whether two match.

Macros

Name	Description
NFFV_MAX_USER_COUNT (see page 26)	The maximum number of users that can be enrolled to a database.

Structs, Records, Enums

	Name	Description
	NffvStatus_ (see page 25)	Enumerates enrollment or verification values of the Nffv.
	NffvStatus (see page 25)	Enumerates enrollment or verification values of the Nffv.

4.1.1.1.1 Functions

The following table lists functions in this documentation.

Functions

	Name	Description
	NffvCancel (see page 14)	Cancels a fingerprint enrollment or verification operation.
	NffvClearUsers (see page 15)	Removes all the users which were enrolled to a database.
	NffvEnroll (see page 15)	Gets a fingerprint from a scanner and saves it to a database.
	NffvFreeMemory (see page 16)	Releases memory allocated by the NffvGetAvailableScannerModules function..
	NffvGetAvailableScannerModulesA (see page 16)	Returns available fingerprint scanner modules for usage in the Free Fingerprint Verification SDK.
	NffvGetAvailableScannerModulesW (see page 16)	Returns available fingerprint scanner modules for usage in the Free Fingerprint Verification SDK.
	NffvGetErrorMessageA (see page 17)	Gets an error message. Use this function for errors handling.
	NffvGetErrorMessageW (see page 17)	Gets an error message. Use this function for errors handling.
	NffvGetInfoA (see page 17)	Retrieves information about the Nffv library.

	NffvGetInfoW (see page 18)	Retrieves information about the library.
	NffvGetMatchingThreshold (see page 18)	Gets the minimum similarity value that verification function uses to determine whether the fingerprint matches.
	NffvGetQualityThreshold (see page 19)	Gets an image quality threshold.
	NffvGetUser (see page 19)	Gets the information from a users list about an enrolled user.
	NffvGetUserById (see page 19)	Returns user details by the Id from a database.
	NffvGetUserCount (see page 20)	Retrieves the number of users enrolled to database.
	NffvGetUserIndexById (see page 20)	Retrieves the index from users list of a user indicated by the Id.
	NffvInitializeA (see page 21)	... more (see page 21)
	NffvInitializeW (see page 21)	... more (see page 21)
	NffvRemoveUser (see page 22)	Removes a user from users list (database).
	NffvSetMatchingThreshold (see page 22)	Sets the minimum similarity value that verification function uses to determine whether the fingerprint matches.
	NffvSetQualityThreshold (see page 23)	Sets an image quality threshold.
	NffvUninitialize (see page 23)	Releases memory resources.
	NffvUserGetHBitmap (see page 23)	Gets a handle to the bitmap of a user fingerprint.
	NffvUserGetId (see page 24)	Gets user Id by specified user details (NffvUser).
	NffvVerify (see page 24)	Compares a captured fingerprint with the one that was enrolled to a database before in order to determine whether two match.

Module

Nffv Unit ([see page 12](#))

4.1.1.1.1.1 NffvCancel Function

Cancels a fingerprint enrollment or verification operation.

C++

```
NResult N_API NffvCancel();
```

Returns

If the function succeeds the return value is N_OK. Otherwise the error code ([see page 49](#)) is returned.

Remarks

This method is useful when the fingerprint enrollment or verification operation take too long. In this case a cancel dialog can be shown for a user to cancel this operation.

Example

This C++ example demonstrates how to stop an enrollment and verification operation:

```
//...
//Function for cancelling enrollment and verification
void OnCancelScan()
{
    NffvCancel();
}
```

Container

File: Nffv.h

Module: Nffv Unit ([see page 12](#))

4.1.1.1.1.2 NffvClearUsers Function

Removes all the users which were enrolled to a database.

C++

```
NResult N_API NffvClearUsers();
```

Returns

If the function succeeds, the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

This functions removes all the users that were enrolled to a database, so be careful when using this function.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.3 NffvEnroll Function

Gets a fingerprint from a scanner and saves it to a database.

C++

```
NResult N_API NffvEnroll(NUInt timeout, NffvStatus * pStatus, HNffvUser * pHUser);
```

Parameters

Parameters	Description
NUInt timeout	[in] Specifies the time in milliseconds after which the fingerprint scanner stops scanning fingerprint. This usually happens when a finger is removed from a scanner for longer than <i>timeout</i> milliseconds.
NffvStatus * pStatus	[out] Enrollment (see page 5) status value enumerated by the NffvStatus (see page 25) enumeration.
pHUser	[out] A pointer to the FFV user object that provides functions for managing enrolled users.

Returns

If the function succeeds the N_OK value is returned. Otherwise, an error code (see page 49) is returned.

Example

This C++ code demonstrates how to enroll a user:

```
UINT CMainForm::EnrollUserThread(LPVVOID pParam)
{
    EnrollParam *pEnrollParam = (EnrollParam*)pParam;
    CMainForm* form = (CMainForm*)pEnrollParam->pMainForm;

    NResult result = NffvEnroll(form->m_iEnrollTimeout, &pEnrollParam->engineStatus,
                                pEnrollParam->pHUser);
    if(NFailed(result)) throw result;

    pEnrollParam->pBusyForm->Stop();

    return 0;
}
```

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.4 NffvFreeMemory Function

Releases memory allocated by the NffvGetAvailableScannerModules function..

C++

```
void N_API NffvFreeMemory(void * pBlock);
```

Parameters

Parameters	Description
void * pBlock	[out] A pointer to memory block that should be released.

Returns

If the functions succeeds the return value is N_OK. Otherwise, the function returns an error code (see page 49).

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.5 NffvGetAvailableScannerModulesA Function

Returns available fingerprint scanner modules for usage in the Free Fingerprint Verification SDK.

C++

```
NResult N_API NffvGetAvailableScannerModulesA(NAChar * * pSzValue);
```

Parameters

Parameters	Description
NAChar * * pSzValue	[out] A string that contains the list of scanners separated by semicolons.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

This function is an ANSI version of the function.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.6 NffvGetAvailableScannerModulesW Function

Returns available fingerprint scanner modules for usage in the Free Fingerprint Verification SDK.

C++

```
NResult N_API NffvGetAvailableScannerModulesW(NWChar * * pSzValue);
```

Parameters

Parameters	Description
NWChar * * pSzValue	[out] A string that contains the list of scanners separated by semicolons.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

This function is a Unicode version of the function.

Container**File:** Nffv.h**Module:** Nffv Unit (see page 12)**4.1.1.1.1.7 NffvGetErrorMessageA Function**

Gets an error message. Use this function for errors handling.

C++

```
NInt N_API NffvGetErrorMessageA(NResult code, NChar * szValue);
```

Parameters

Parameters	Description
NResult code	[in] An error code.
NChar * szValue	[out] Pointer to memory block that contains an error description.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

This function is an ANSI version of the function.

Container**File:** Nffv.h**Module:** Nffv Unit (see page 12)**4.1.1.1.1.8 NffvGetErrorMessageW Function**

Gets an error message. Use this function for errors handling.

C++

```
NInt N_API NffvGetErrorMessageW(NResult code, NWChar * szValue);
```

Parameters

Parameters	Description
NResult code	[in] An error code.
NWChar * szValue	[out] Pointer to memory block that contains an error description.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

This function is a Unicode version of the function.

Container**File:** Nffv.h**Module:** Nffv Unit (see page 12)**4.1.1.1.1.9 NffvGetInfoA Function**

Retrieves information about the Nffv library.

C++

```
NResult N_API NffvGetInfoA(NLibraryInfoA * pValue);
```

Parameters

Parameters	Description
NLibraryInfoA * pValue	[out] Pointer to NLibraryInfoA structure that receives library information.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

This function is an ANSI version of the function.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.10 NffvGetInfoW Function

Retrieves information about the library.

C++

```
NResult N_API NffvGetInfoW(NLibraryInfoW * pValue);
```

Parameters

Parameters	Description
NLibraryInfoW * pValue	[out] Pointer to NLibraryInfoA structure that receives library information.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

This function is a Unicode version of the function.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.11 NffvGetMatchingThreshold Function

Gets the minimum similarity value that verification function uses to determine whether the fingerprint matches.

C++

```
NResult N_API NffvGetMatchingThreshold(NInt * pValue);
```

Parameters

Parameters	Description
NInt * pValue	[out] Similarity value (matching threshold) for the Nffv. Values are in range [0, MaxInt]. MaxInt is a maximum integer value.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

For more information about the matching threshold, please read chapter Matching Threshold (see page 6).

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.12 NffvGetQualityThreshold Function

Gets an image quality threshold.

C++

```
NResult N_API NffvGetQualityThreshold(NByte * pValue);
```

Parameters

Parameters	Description
NByte * pValue	[out] Quality threshold. The value is in range [0, 255].

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

For more information about the quality threshold, please read chapter Quality Threshold (see page 6).

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.13 NffvGetUser Function

Gets the information from a users list about an enrolled user.

C++

```
NResult N_API NffvGetUser(NInt index, HNffvUser * pValue);
```

Parameters

Parameters	Description
NInt index	[in] An index of a user who was enrolled to a database.
HNffvUser * pValue	[out] Information about an enrolled user.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

To get an index of a user you can use NffvGetUserIndexById (see page 20) function.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.14 NffvGetUserById Function

Returns user details by the Id from a database.

C++

```
NResult N_API NffvGetUserById(NInt id, HNffvUser * pValue);
```

Parameters

Parameters	Description
NInt id	[in] User's identification number in a database. This Id is always unique.
HNffvUser * pValue	[out] Information about a user who was enrolled to a database.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.15 NffvGetUserCount Function

Retrieves the number of users enrolled to database.

C++

```
NResult N_API NffvGetUserCount(NInt * pValue);
```

Parameters

Parameters	Description
NInt * pValue	[out] The number of enrolled users.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.16 NffvGetUserIndexById Function

Retrieves the index from users list of a user indicated by the Id.

C++

```
NResult N_API NffvGetUserIndexById(NInt id, NInt * pValue);
```

Parameters

Parameters	Description
NInt id	[in] The user Id. This Id is used in a users database.
NInt * pValue	[out] An index of a user.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.17 NffvInitializeA Function

Initializes the FFV. This function as a parameters takes a name and a password of a previously created database or creates a new database.

C++

```
NResult N_API NffvInitializeA(const NChar * szDbName, const NChar * szPassword, const
NChar * szScannerModules);
```

Parameters

Parameters	Description
const NChar * szDbName	[out] The name of a database. This database will be used to save user fingerprints. The database will be saved to a working folder (or other folder) as a file.
const NChar * szPassword	[out] A database password. If you don't want to protect a database by password, use a blank a password.
const NChar * szScannerModules	[out] A list of scanner modules that should be loaded. It is a list of fingerprint scanners that you will use in your application. If the value is an empty string then no scanners are loaded. If the value is null all scanner modules are loaded. Each scanner module in a list should be separated by a semicolon.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

This function is an ANSI version of the function.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.18 NffvInitializeW Function

Initializes the FFV. This function as a parameters takes a name and a password of a previously created database or creates a new database.

C++

```
NResult N_API NffvInitializeW(const NWChar * szDbName, const NWChar * szPassword, const
NWChar * szScannerModules);
```

Parameters

Parameters	Description
const NWChar * szDbName	[out] The name of a database. This database will be used to save user fingerprints. The database will be saved to a working folder (or other folder) as a file.
const NWChar * szPassword	[out] A database password. If you don't want to protect a database by password, use a blank a password.
const NWChar * szScannerModules	[out] A list of scanner modules that should be loaded. It is a list of fingerprint scanners that you will use in your application. If the value is an empty string then no scanners are loaded. If the value is null all scanner modules are loaded. Each scanner module in a list should be separated by a semicolon.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

This function is Unicode version of the function.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.19 NffvRemoveUser Function

Removes a user from users list (database).

C++

```
NResult N_API NffvRemoveUser(NInt index);
```

Parameters

Parameters	Description
NInt index	[in] An index number of a user that should be removed from a list.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

All enrolled users during the execution of an application are loaded from a database to a list.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.20 NffvSetMatchingThreshold Function

Sets the minimum similarity value that verification function uses to determine whether the fingerprint matches.

C++

```
NResult N_API NffvSetMatchingThreshold(NInt value);
```

Parameters

Parameters	Description
NInt value	[in] Similarity value (matching threshold) to set.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

For more information about the matching threshold, please read chapter Matching Threshold (see page 6).

The default matching threshold value is 48.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.21 NffvSetQualityThreshold Function

Sets an image quality threshold.

C++

```
NResult N_API NffvSetQualityThreshold(NByte value);
```

Parameters

Parameters	Description
NByte value	[in] Quality threshold to set.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Remarks

For more information about the quality threshold, please read chapter Quality Threshold (see page 6).

The default matching threshold value is 100.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.22 NffvUninitialize Function

Releases memory resources.

C++

```
void N_API NffvUninitialize();
```

Returns

If the functions succeeds the return value is N_OK. Otherwise, the function returns an error code (see page 49).

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.23 NffvUserGetHBitmap Function

Gets a handle to the bitmap of a user fingerprint.

C++

```
NResult N_API NffvUserGetHBitmap(HNffvUser hUser, NHandle * phBitmap);
```

Parameters

Parameters	Description
HNffvUser hUser	[in] A handle to NffvUser (see page 25) object which is used to manage users.
phBitmap	[out] A handle to a bitmap of the last scanned fingerprint.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.24 NffvUserGetId Function

Gets user Id by specified user details (NffvUser).

C++

```
NResult N_API NffvUserGetId(HNffvUser hUser, NInt * pValue);
```

Parameters

Parameters	Description
HNffvUser hUser	[in] Handle to NffvUser.
NInt * pValue	[out] User Id.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Container

File: Nffv.h

Module: Nffv Unit (see page 12)

4.1.1.1.1.25 NffvVerify Function

Compares a captured fingerprint with the one that was enrolled to a database before in order to determine whether two match.

C++

```
NResult N_API NffvVerify(HNffvUser hUser, NUInt timeout, NffvStatus * pStatus, NInt * pScore);
```

Parameters

Parameters	Description
HNffvUser hUser	[in] A handle to a database record that should be matched with the scanned fingerprint.
NUInt timeout	[in] Specifies the time in milliseconds after which the fingerprint scanner stops scanning fingerprint. This usually happens when a finger is removed from a scanner for longer than <i>timeout</i> milliseconds.
NffvStatus * pStatus	[out] One of the verification status values enumerated in NffvStatus (see page 25).
NInt * pScore	[out] A matching score of two fingerprints verification.

Returns

If the function succeeds the return value is N_OK. Otherwise, an error code (see page 49) is returned.

Example

This C++ example demonstrates how to verify two fingerprints:

```
UINT CMainForm::VerifyUserThread(LPVOID pParam)
{
    VerifyParam *pVerifyParam = (VerifyParam*)pParam;
    CMainForm* form = ((CMainForm*)pVerifyParam->pMainForm);

    NResult result = NffvVerify(pVerifyParam->hUser, 20000,
                               &pVerifyParam->engineStatus, pVerifyParam->pScore);
    if(NFailed(result)) throw result;

    pVerifyParam->pBusyForm->Stop();

    return 0;    // thread completed successfully
}
```

Container**File:** Nffv.h**Module:** Nffv Unit (see page 12)**4.1.1.1.2 Structs, Records, Enums**

The following table lists structs, records, enums in this documentation.

Enumerations

	Name	Description
	NffvStatus_ (see page 25)	Enumerates enrollment or verification values of the Nffv.
	NffvStatus (see page 25)	Enumerates enrollment or verification values of the Nffv.

Module

Nffv Unit (see page 12)

4.1.1.1.2.1 NffvStatus_ Enumeration

Enumerates enrollment or verification values of the Nffv.

C++

```
enum NffvStatus_ {  
    nfesTemplateCreated = 1,  
    nfesNoScanner = 2,  
    nfesScannerTimeout = 3,  
    nfesUserCanceled = 4,  
    nfesQualityCheckFailed = 100  
};
```

Members

Members	Description
nfesTemplateCreated = 1	Indicates that the fingerprint template was created.
nfesNoScanner = 2	Indicates that there is no fingerprint scanner connected.
nfesScannerTimeout = 3	Indicates that the fingerprint scanner has reached the timeout.
nfesUserCanceled = 4	Indicates that a user has canceled a fingerprint scanning.
nfesQualityCheckFailed = 100	Indicates that the Free Fingerprint Verification SDK had failed to check the quality of a fingerprint.

Container**File:** Nffv.h**Module:** Nffv Unit (see page 12)**4.1.1.1.2.2 NffvStatus Enumeration**

Enumerates enrollment or verification values of the Nffv.

C++

```
typedef enum NffvStatus_ {  
    nfesTemplateCreated = 1,  
    nfesNoScanner = 2,  
    nfesScannerTimeout = 3,  
    nfesUserCanceled = 4,  
    nfesQualityCheckFailed = 100  
} NffvStatus;
```

Members

Members	Description
nfesTemplateCreated = 1	Indicates that the fingerprint template was created.
nfesNoScanner = 2	Indicates that there is no fingerprint scanner connected.
nfesScannerTimeout = 3	Indicates that the fingerprint scanner has reached the timeout.
nfesUserCanceled = 4	Indicates that a user has canceled a fingerprint scanning.
nfesQualityCheckFailed = 100	Indicates that the Free Fingerprint Verification SDK had failed to check the quality of a fingerprint.

Container**File:** Nffv.h**Module:** Nffv Unit (see page 12)

4.1.1.1.3 Macros

The following table lists macros in this documentation.

Macros

Name	Description
NFFV_MAX_USER_COUNT (see page 26)	The maximum number of users that can be enrolled to a database.

Module

Nffv Unit (see page 12)

4.1.1.1.3.1 NFFV_MAX_USER_COUNT Macro

The maximum number of users that can be enrolled to a database.

C++

```
#define NFFV_MAX_USER_COUNT 10
```

Notes

You can not change this value.

Container**File:** Nffv.h**Module:** Nffv Unit (see page 12)

4.2 .NET Reference

This chapter provides the Free Fingerprint Verification SDK programming reference for Microsoft .NET framework.

Remarks

If you are developing your own application using one of a .NET programming language, you should include this dynamic-link library into your biometric solution project:

- Neurotec.Biometrics.Nffv.dll (contains enrollment and verification methods). This Dll is a wrapper of the Nffv.dll.

Nffv.dll file should be located in the same folder as Neurotec.Biometrics.Nffv.dll. Also NffvServer.exe is required for using Neurotec.Biometrics.Nffv.dll.

Namespaces

Name	Description
Neurotec.Biometrics (see page 27)	Contains classes and methods that provide the Free Fingerprint Verification SDK functionality.

4.2.1 Neurotec.Biometrics Namespace

Contains classes and methods that provide the Free Fingerprint Verification SDK functionality.

Module

.NET Reference ([see page 26](#))

Classes

	Name	Description
	Nffv (see page 27)	The main class of the Free Fingerprint Verification SDK. Provides methods and properties for working with user collection and enrolling or verifying user fingerprints.
	NffvUser (see page 34)	Provides methods and properties for working with users.

Structs, Records, Enums

	Name	Description
	NffvStatus (see page 35)	Enumerates enrollment or verification status values.

4.2.1.1 Classes

The following table lists classes in this documentation.

Classes

	Name	Description
	Nffv (see page 27)	The main class of the Free Fingerprint Verification SDK. Provides methods and properties for working with user collection and enrolling or verifying user fingerprints.
	NffvUser (see page 34)	Provides methods and properties for working with users.

4.2.1.1.1 Nffv Class

The main class of the Free Fingerprint Verification SDK. Provides methods and properties for working with user collection and enrolling or verifying user fingerprints.

C#

```
public class Nffv : MarshalByRefObject, IDisposable;
```

Class Hierarchy

Methods

	Name	Description
	Nffv (see page 28)	Initializes a new instance of the Nffv class. During the initialization a new database is created or used previously created.

Nffv Classes

	Name	Description
	UserCollection (see page 29)	Represents a collection of NffvUsers objects that represent the user fingerprints enrolled to a database.

Nffv Fields

	Name	Description
	DllName (see page 31)	The name of a dynamic-linked library which contains unmanaged functionality of the Free Fingerprint Verification SDK.
	MaxUserCount (see page 31)	The maximum number of users that can be enrolled to a database.

Nffv Methods

	Name	Description
	Cancel (see page 32)	Cancels a fingerprint enrollment or verification operation.
	Dispose (see page 32)	Disposes resources used by the Nffv.
	Enroll (see page 32)	Gets a fingerprint from a scanner and saves it to a database.
 	GetAvailableScannerModules (see page 33)	Returns available fingerprint scanner modules for usage in the Free Fingerprint Verification SDK.
	GetUserById (see page 33)	Returns a user details by the Id from the UserCollection (see page 29).
	Verify (see page 33)	Compares a captured fingerprint with the one that was enrolled to a database before in order to determine whether two match.

Nffv Properties

	Name	Description
	MatchingThreshold (see page 34)	Gets or sets the minimum similarity value that verification method uses to determine whether the fingerprint matches.
	QualityThreshold (see page 34)	Gets or sets image quality threshold.

Container

File: Nffv.cs

Namespace: Neurotec.Biometrics ([see page 27](#))

Module: .NET Reference ([see page 26](#))

4.2.1.1.1.1 Nffv Constructor**4.2.1.1.1.1.1 Nffv.Nffv Constructor (string, string)**

Initializes a new instance of the Nffv class. During the initialization a new database is created or used previously created.

C#

```
public Nffv(string dbName, string password);
```

Parameters

Parameters	Description
string dbName	A name of database. This database will be used to save user fingerprints. The database will be saved to a working folder as a file.
string password	A database password. If you don't want to protect a database by password, use an empty string as a password.

Example

This C# example code demonstrates how to create a new instance of the Nffv class.

```
string dbName = "FingerprintsDatabase.dat";
string password = "passwd";

Neurotec.Biometrics.Nffv engine = null;

//Creates a new instance of the Nffv class
engine = new Neurotec.Biometrics.Nffv(dbName, password);
```

The same example code for VB.NET:

```
Dim dbName As String = "FingerprintsDatabase.dat"
Dim password As String = "passwd"

Dim engine As Global.Neurotec.Biometrics.Nffv = Nothing

engine = New Global.Neurotec.Biometrics.Nffv(dbName, password)
```

4.2.1.1.1.2 Nffv.Nffv Constructor (string, string, string)

Initializes a new instance of the Nffv class. During the initialization a new database is created or used previously created.

C#

```
public Nffv(string dbName, string password, string scannerModules);
```

Parameters

Parameters	Description
string dbName	A name of database. This database will be used to save user fingerprints. The database will be saved to a working folder as a file.
string password	A database password. If you don't want to protect a database by password, use an empty string as a password.
string scannerModules	A list of scanner modules that should be loaded. It is a list of fingerprint scanners that you will use in your application. Each fingerprint scanner's name in the scanner module is separated by semicolon.

Remarks

For the list of available fingerprint scanners see a chapter Supported Scanners.

Example

This C# example code demonstrates how to create a new instance of the Nffv calss.

```
string dbName = "FingerprintsDatabase.dat";
string password = "passwd";
string scanners = "Upek;Futronic";

Neurotec.Biometrics.Nffv engine = null;

//Creates a new instance of the Nffv class
engine = new Neurotec.Biometrics.Nffv(dbName, password, scanners);
```

The same example code for VB.NET:

```
Dim dbName As String = "FingerprintsDatabase.dat"
Dim password As String = "passwd"
Dim scanners As String = "Upek;Futronic"

Dim engine As Global.Neurotec.Biometrics.Nffv = Nothing

engine = New Global.Neurotec.Biometrics.Nffv(dbName, password, scanners)
```

4.2.1.1.1.2 Nffv Classes

4.2.1.1.1.2.1 Nffv.UserCollection Class

Represents a collection of NffvUsers objects that represent the user fingerprints enrolled to a database.

C#

```
[Serializable]
public sealed class UserCollection : CollectionBase;
```

Class Hierarchy

Notes

This class is a sealed class, so it has a limited extensibility (other classes cannot inherit from it).

UserCollection Methods

	Name	Description
	Add ( see page 30)	Adds a user to a UserCollection.
	Contains ( see page 30)	Returns a Boolean value indicating whether a UserCollection object contains an element with a specified key.
	IndexOf ( see page 31)	Returns an index of the UserCollection item specified by Id.

Container

File: Nffv.cs

Namespace: Neurotec.Biometrics ( see page 27)

Module: .NET Reference ( see page 26)

4.2.1.1.1.2.1.1 UserCollection Methods

4.2.1.1.1.2.1.1.1 Nffv.UserCollection.Add Method

Adds a user to a UserCollection ( see page 29).

C#

```
internal NffvUser Add(IntPtr hUser);
```

Parameters

Parameters	Description
IntPtr hUser	A reference to an object that represents a user which should be added to a collection.

Example

To add a user to database you can use this C# code:

```
public class UserEnrollment
{
    UserDatabase _userDB;
    _userDB.Add(new UserRecord(engineUser.Id, userName));
}

public class UserRecord
{
    //...
    public UserRecord(int id, string name)
    {
        _id = id;
        _name = name;
    }
}
```

4.2.1.1.1.2.1.1.2 Nffv.UserCollection.Contains Method

Returns a Boolean value indicating whether a UserCollection ( see page 29) object contains an element with a specified key.

C#

```
public bool Contains(int id);
```

Parameters

Parameters	Description
int id	An integer value that specifies the Id for which to search the element of the collection.

Returns

A Boolean value indicating whether the UserCollection (see page 29) contains an elements with the specified Id.

If the return value is True, the collection contains an element with an Id specified. Otherwise, the return value is False.

Example

This C# example demonstrates how to use this method:

```
int id = 3;

if UserCollection.Contains(id)
    MsgBox("The desired user is in collection");
else
    MsgBox("The desired user was not find in a collection");
```

The VB.NET code this method:

```
Dim id As Integer = 3

If UserCollection.Contains(id) Then
    MsgBox("The desired user is in the collection.")
Else
    MsgBox("The desired user was not find in the collection.")
End If
```

4.2.1.1.1.2.1.1.3 Nffv.UserCollection.IndexOf Method

Returns an index of the UserCollection (see page 29) item specified by Id.

C#

```
public int IndexOf(int id);
```

Parameters

Parameters	Description
int id	The Id of a user to search in a collection.

Returns

A collection index of a user specified by Id.

4.2.1.1.1.3 Nffv Fields**4.2.1.1.1.3.1 Nffv.DllName Field**

The name of a dynamic-linked library which contains unmanaged functionality of the Free Fingerprint Verification SDK.

C#

```
public const string DllName = "Nffv.dll";
```

4.2.1.1.1.3.2 Nffv.MaxUserCount Field

The maximum number of users that can be enrolled to a database.

C#

```
public const int MaxUserCount = 10;
```

Remarks

You can add up-to 10 users to a database.

4.2.1.1.1.4 Nffv Methods

4.2.1.1.1.4.1 Nffv.Cancel Method

Cancels a fingerprint enrollment or verification operation.

C#

```
public void Cancel();
```

Remarks

This method is useful when the fingerprint enrollment or verification operation take too long. In this case a message box can be shown for a user to cancel this operation.

Example

This C# code demonstrates how to cancel enrollment or verification operation:

```
Nffv engine;  
engine.Cancel();
```

The same code using VB.NET notation:

```
Private engine As Nffv  
engine.Cancel()
```

4.2.1.1.1.4.2 Nffv.Dispose Method

Disposes resources used by the Nffv (see page 27).

C#

```
public void Dispose();
```

4.2.1.1.1.4.3 Nffv.Enroll Method

Gets a fingerprint from a scanner and saves it to a database.

C#

```
public NffvUser Enroll(uint timeout, out NffvStatus status);
```

Parameters

Parameters	Description
uint timeout	Specifies the time in milliseconds after which the fingerprint scanner stops scanning fingerprint. This usually happens when a finger is removed from a scanner for longer than <i>timeout</i> milliseconds.
out NffvStatus status	Enrollment (see page 5) status value indicated by one of the value enumerated in NffvStatus (see page 35).

Returns

A reference to NffvUser (see page 34) object which provides methods for managing enrolled users.

If there were problem enrolling a fingerprint, the method returns a zero pointer.

Example

This C# example demonstrates the usage of the *Enroll* method:

```
//Field that holds a reference to Nffv object  
Nffv engine;  
  
//Internal class that saves the result of fingerprint enrolment  
internal class EnrollmentResult  
{  
    public NffvStatus engineStatus;
```

```
    public NffvUser engineUser;
};

//Method used for a fingerprint enrollment
public void doEnroll(object sender, DoWorkEventArgs args)
{
    EnrollmentResult enrollmentResults = new EnrollmentResult();
    enrollmentResults.engineUser = engine.Enroll(20000, out enrollmentResults.engineStatus);
    args.Result = enrollmentResults;
}
```

4.2.1.1.1.4.4 Nffv.GetAvailableScannerModules Method

Returns available fingerprint scanner modules for usage in the Free Fingerprint Verification SDK.

C#

```
public static string GetAvailableScannerModules();
```

Returns

A string that contains the list of scanners separated by semicolons.

4.2.1.1.1.4.5 Nffv.GetUserById Method

Returns a user details by the Id from the UserCollection ([see page 29](#)).

C#

```
public NffvUser GetUserById(int id);
```

Parameters

Parameters	Description
int id	User's identification number in a collection.

Returns

A reference to the NffvUser ([see page 34](#)) object that contains an information about a user indicated by Id.

4.2.1.1.1.4.6 Nffv.Verify Method

Compares a captured fingerprint with the one that was enrolled to a database before in order to determine whether two match.

C#

```
public int Verify(NffvUser user, uint timeout, out NffvStatus status);
```

Parameters

Parameters	Description
NffvUser user	A reference to a database record that should be matched with the scanned fingerprint.
uint timeout	Specifies the time in milliseconds after which the fingerprint scanner stops scanning fingerprint. This usually happens when a finger is removed from a scanner for longer than <i>timeout</i> milliseconds.
out NffvStatus status	The verification status value indicated by one of the value enumerated in NffvStatus (see page 35).

Returns

This function returns a matching score.

Example

This C# sample code demonstrates how to verify two fingerprints.

```
Nffv engine;
```

```
//An internal class that saves the verification result
internal class VerificationResult
{
    public NffvStatus engineStatus;
    public int score;
};

public void doVerify(object sender, DoWorkEventArgs args)
{
    VerificationResult verificationResult = new VerificationResult();
    verificationResult.score = engine.Verify((NffvUser)args.Argument, 20000, out
    verificationResult.engineStatus);
    args.Result = verificationResult;
}
```

Note that it isn't a complete code that can be used in your application.

For a complete code see the C# Sample application.

4.2.1.1.1.5 Nffv Properties

4.2.1.1.1.5.1 Nffv.MatchingThreshold Property

Gets or sets the minimum similarity value that verification method uses to determine whether the fingerprint matches.

C#

```
public int MatchingThreshold;
```

Property value

The minimum similarity value that verification function accept for the same finger fingerprints. The default value is 0.01 %.

4.2.1.1.1.5.2 Nffv.QualityThreshold Property

Gets or sets image quality threshold.

C#

```
public byte QualityThreshold;
```

Property value

The fingerprint quality threshold. The value should be in range [0, 255]. The default value is 100.

4.2.1.1.2 NffvUser Class

Provides methods and properties for working with users.

C#

```
public sealed class NffvUser : MarshalByRefObject;
```

Class Hierarchy

NffvUser Methods

	Name	Description
	GetBitmap (see page 35)	Returns the bitmap of the last scanned fingerprint.
	GetHBitmap (see page 35)	Returns a handle to the bitmap of the last scanned fingerprint.

Container

File: NffvUser.cs

Namespace: Neurotec.Biometrics ([see page 27](#))

Module: .NET Reference ([see page 26](#))

4.2.1.1.2.1 NffvUser Methods

4.2.1.1.2.1.1 NffvUser.GetBitmap Method

Returns the bitmap of the last scanned fingerprint.

```
C#
public Bitmap GetBitmap();
```

Returns
A Bitmap object.

4.2.1.1.2.1.2 NffvUser.GetHBitmap Method

Returns a handle to the bitmap of the last scanned fingerprint.

```
C#
public IntPtr GetHBitmap();
```

Returns
A pointer to Bitmap object.

4.2.1.2 Structs, Records, Enums

The following table lists structs, records, enums in this documentation.

Enumerations

	Name	Description
	NffvStatus (see page 35)	Enumerates enrollment or verification status values.

4.2.1.2.1 Neurotec.Biometrics.NffvStatus Enumeration

Enumerates enrollment or verification status values.

```
C#
[Serializable]
public enum NffvStatus {
    TemplateCreated = 1,
    NoScanner = 2,
    ScannerTimeout = 3,
    UserCanceled = 4,
    QualityCheckFailed = 100
}
```

Members

Members	Description
TemplateCreated = 1	Indicates that the fingerprint template was created.
NoScanner = 2	Indicates that there is no fingerprint scanner connected.
ScannerTimeout = 3	Indicates that the fingerprint scanner has reached the timeout.
UserCanceled = 4	Indicates that a user has canceled a fingerprint scanning.
QualityCheckFailed = 100	Indicates that the Free Fingerprint Verification SDK had failed to check the quality of a fingerprint.

Container

File: Nffv.cs

Namespace: Neurotec.Biometrics ([see page 27](#))

Module: .NET Reference ([see page 26](#))

4.3 Java Reference

This chapter provides the Free Fingerprint Verification SDK programming reference for Java programming language.

Notes

A 64-bit JVM needs to be used to execute NffvSample.jar.

Packages

Name	Description
com.neurotechnology.Nffv (see page 36)	Classes under this namespace provides methods for the com.neurotechnology.Nffv (see page 37) library.

4.3.1 com.neurotechnology.Nffv Package

Classes under this namespace provides methods for the com.neurotechnology.Nffv ([see page 37](#)) library.

Module

Java Reference ([see page 36](#))

Classes

	Name	Description
	Nffv (see page 37)	The main class of the Free Fingerprint Verification SDK. Provides methods and properties for working with users list and enrolling or verifying user fingerprints.
	NffvImage (see page 41)	Provides methods for managing images.
	NffvUser (see page 44)	Provides methods for working with users.
	ScannerModule (see page 45)	Provides methods for setting and getting scanner names from the ScannerModule.

Structs, Records, Enums

	Name	Description
	NffvStatus (see page 46)	Enumerates NffvStatus values.

4.3.1.1 Classes

The following table lists classes in this documentation.

Classes

	Name	Description
	Nffv (see page 37)	The main class of the Free Fingerprint Verification SDK. Provides methods and properties for working with users list and enrolling or verifying user fingerprints.
	NffvImage (see page 41)	Provides methods for managing images.
	NffvUser (see page 44)	Provides methods for working with users.
	ScannerModule (see page 45)	Provides methods for setting and getting scanner names from the ScannerModule.

4.3.1.1.1 Nffv Class

The main class of the Free Fingerprint Verification SDK. Provides methods and properties for working with users list and enrolling or verifying user fingerprints.

Class Hierarchy

Java

```
public class Nffv;
```

Methods

	Name	Description
	Nffv (see page 37)	Creates a new instance of the Nffv.

Nffv Methods

	Name	Description
	cancel (see page 38)	Cancels current action.
	clearUsers (see page 38)	Removes all users from a database.
	contains (see page 38)	Checks if the database contains a concrete user.
	enroll (see page 38)	Gets a fingerprint from a scanner and saves it to a database.
	finalize (see page 38)	Implements standard Java method used by the garbage collector.
	getAvailableScannerModules (see page 39)	Returns available fingerprint scanner modules for usage in the Free Fingerprint Verification SDK.
	getEngineStatus (see page 39)	Gets status information of the com.neurotechnology.Nffv (see page 36).
	getMatchingThreshold (see page 39)	Gets the minimum similarity value that verification function uses to determine whether the fingerprint matches.
	getMaxUserCount (see page 39)	The maximum number of users that can be enrolled to a database.
	getQualityThreshold (see page 39)	Gets image quality threshold.
	getUserById (see page 39)	Returns a user details by the Id.
	getUsers (see page 40)	Gets a list of users enrolled to a database.
	removeUser (see page 40)	Removes a concrete user from a database.
	removeUserID (see page 40)	Removes user's ID.
	setEngineStatus (see page 40)	Sets status information for the com.neurotechnology.Nffv (see page 36).
	setIntEngineStatus (see page 40)	Sets status information for the com.neurotechnology.Nffv (see page 36).
	setMatchingThreshold (see page 41)	Sets the minimum similarity value that verification function uses to determine whether the fingerprint matches.
	setQualityThreshold (see page 41)	Sets image quality threshold.
	verify (see page 41)	Compares a captured fingerprint with the one that was enrolled to a database before in order to determine whether two match.

Container

File: Nffv.java

Package: com.neurotechnology.Nffv ([see page 36](#))

Module: Java Reference ([see page 36](#))

4.3.1.1.1.1 Nffv.Nffv Constructor

Creates a new instance of the Nffv.

Parameters

Parameters	Description
String database	Name of a database,
String password	Password for a database.
ScannerModule[] scannerModules	List of scanner modules that should be loaded.

Java

```
public Nffv(String database, String password, ScannerModule[] scannerModules);
```

4.3.1.1.1.2 Nffv Methods**4.3.1.1.1.2.1 Nffv.cancel Method**

Cancels current action.

Java

```
public void cancel();
```

4.3.1.1.1.2.2 Nffv.clearUsers Method

Removes all users from a database.

Java

```
public void clearUsers();
```

4.3.1.1.1.2.3 Nffv.contains Method

Checks if the database contains a concrete user.

Parameters

Parameters	Description
NffvUser user	User details to check.

Returns

Boolean value indicating if a database contains a concrete user.

Java

```
public boolean contains(NffvUser user);
```

4.3.1.1.1.2.4 Nffv.enroll Method

Gets a fingerprint from a scanner and saves it to a database.

Parameters

Parameters	Description
int timeout	Specifies the time in milliseconds after which the fingerprint scanner stops scanning fingerprint. This usually happens when a finger is removed from a scanner for longer than timeout milliseconds.

Returns

NffvUser (see page 44) object that contains (see page 38) details of an enrolled user.

Java

```
public NffvUser enroll(int timeout);
```

4.3.1.1.1.2.5 Nffv.finalize Method

Implements standard Java method used by the garbage collector.

Java

```
public void finalize() throws Throwable;
```

4.3.1.1.1.2.6 Nffv.getAvailableScannerModules Method

Returns available fingerprint scanner modules for usage in the Free Fingerprint Verification SDK.

Returns

Array that contains (see page 38) available scanner modules.

Java

```
public static ScannerModule getAvailableScannerModules();
```

4.3.1.1.1.2.7 Nffv.getEngineStatus Method

Gets status information of the com.neurotechnology.Nffv (see page 36).

Returns

NffvStatus (see page 46) object that holds information about Nffv (see page 37).

Java

```
public NffvStatus getEngineStatus();
```

4.3.1.1.1.2.8 Nffv.getMatchingThreshold Method

Gets the minimum similarity value that verification function uses to determine whether the fingerprint matches.

Returns

The minimum similarity value that verification function accept for the same finger fingerprints. The default value is 0.01 %.

Java

```
public int getMatchingThreshold();
```

4.3.1.1.1.2.9 Nffv.getMaxUserCount Method

The maximum number of users that can be enrolled to a database.

Returns

The maximum number of users that can be enrolled to a database.

Java

```
public int getMaxUserCount();
```

4.3.1.1.1.2.10 Nffv.getQualityThreshold Method

Gets image quality threshold.

Returns

Returns fingerprint quality threshold. The value should be in range [0, 255]. The default value is 100.

Java

```
public int getQualityThreshold();
```

4.3.1.1.1.2.11 Nffv.getUserByID Method

Returns a user details by the Id.

Parameters

Parameters	Description
int id	User Id.

Returns

NffvUser (see page 44) object that contains (see page 38) user details.

Java

```
public NffvUser getUserByID(int id);
```

4.3.1.1.1.2.12 Nffv.getUsers Method

Gets a list of users enrolled to a database.

Returns

List of users that was enrolled to a database.

Java

```
public List<NffvUser> getUsers();
```

4.3.1.1.1.2.13 Nffv.removeUser Method

Removes a concrete user from a database.

Parameters

Parameters	Description
NffvUser user	NffvUser (see page 44) object that should be removed.

Java

```
public void removeUser(NffvUser user) throws Exception;
```

4.3.1.1.1.2.14 Nffv.removeUserID Method

Removes user's ID.

Parameters

Parameters	Description
int ID	User's ID to remove.

Java

```
public void removeUserID(int ID);
```

4.3.1.1.1.2.15 Nffv.setEngineStatus Method

Sets status information for the com.neurotechnology.Nffv (see page 36).

Parameters

Parameters	Description
NffvStatus engineStatus	NffvStatus (see page 46) object that holds information to set.

Java

```
protected void setEngineStatus(NffvStatus engineStatus);
```

4.3.1.1.1.2.16 Nffv.setIntEngineStatus Method

Sets status information for the com.neurotechnology.Nffv (see page 36).

Parameters

Parameters	Description
int engineStatus	Number that indicates Nffv (see page 37) status.

Java

```
protected void setIntEngineStatus(int engineStatus);
```

4.3.1.1.1.2.17 Nffv.setMatchingThreshold Method

Sets the minimum similarity value that verification function uses to determine whether the fingerprint matches.

Parameters

Parameters	Description
int value	The minimum similarity value that verification function accept for the same finger fingerprints.

Java

```
public void setMatchingThreshold(int value);
```

4.3.1.1.1.2.18 Nffv.setQualityThreshold Method

Sets image quality threshold.

Parameters

Parameters	Description
Image	quality threshold to set.

Java

```
public void setQualityThreshold(int value);
```

4.3.1.1.1.2.19 Nffv.verify Method

Compares a captured fingerprint with the one that was enrolled to a database before in order to determine whether two match.

Parameters

Parameters	Description
NffvUser user	A reference to a database record that should be matched with the scanned fingerprint.
int timeout	Specifies the time in milliseconds after which the fingerprint scanner stops scanning fingerprint. This usually happens when a finger is removed from a scanner for longer than timeout milliseconds.

Returns

Matching score.

Java

```
public int verify(NffvUser user, int timeout);
```

4.3.1.1.2 NffvImage Class

Provides methods for managing images.

Class Hierarchy

Java

```
public class NffvImage;
```

NffvImage Methods

	Name	Description
⇒	getBufferedImage (see page 42)	Gets a buffered image.
⇒	getHeight (see page 42)	Retrieves image height.
⇒	getHorizontalResolution (see page 42)	Gets the horizontal resolution of the image.
⇒	getImageData (see page 42)	Gets an image data as a byte array.

	<code>getImageIcon</code> (see page 43)	Get an image icon.
	<code>getStride</code> (see page 43)	Gets the stride (size of one row) of the image.
	<code>getVerticalResolution</code> (see page 43)	Retrieves the vertical resolution of the image.
	<code>getWidth</code> (see page 43)	Retrieves image width.
	<code>setHeight</code> (see page 43)	Sets image height.
	<code>setHorizontalResolution</code> (see page 43)	Sets the horizontal resolution of the image.
	<code>setImageData</code> (see page 44)	Creates an image from a byte array.
	<code>setStride</code> (see page 44)	Sets the stride (size of one row) of the image.
	<code>setVerticalResolution</code> (see page 44)	Sets the vertical resolution of the image.
	<code>setWidth</code> (see page 44)	Sets image width.

Container

File: NffvImage.java

Package: com.neurotechnology.Nffv ([see page 36](#))

Module: Java Reference ([see page 36](#))

4.3.1.1.2.1 NffvImage Methods

4.3.1.1.2.1.1 NffvImage.getBufferedImage Method

Gets a buffered image.

Returns

Buffered image which contains an accessible buffer of the image.

Java

```
public BufferedImage getBufferedImage();
```

4.3.1.1.2.1.2 NffvImage.getHeight Method

Retrieves image height.

Returns

Image height in pixels.

Java

```
public int getHeight();
```

4.3.1.1.2.1.3 NffvImage.getHorizontalResolution Method

Gets the horizontal resolution of the image.

Returns

Horizontal resolution of image.

Java

```
public float getHorizontalResolution();
```

4.3.1.1.2.1.4 NffvImage.getImageData Method

Gets an image data as a byte array.

Returns

Byte array that contains image data.

Java

```
public byte getImageData();
```

4.3.1.1.2.1.5 NffvImage.getImageIcon Method

Get an image icon.

Returns

An icon of the image.

Java

```
public ImageIcon getImageIcon();
```

4.3.1.1.2.1.6 NffvImage.getStride Method

Gets the stride (size of one row) of the image.

Returns

Image stride.

Java

```
public int getStride();
```

4.3.1.1.2.1.7 NffvImage.getVerticalResolution Method

Retrieves the vertical resolution of the image.

Returns

Vertical resolution of image.

Java

```
public float getVerticalResolution();
```

4.3.1.1.2.1.8 NffvImage.getWidth Method

Retrieves image width.

Returns

Image width in pixels.

Java

```
public int getWidth();
```

4.3.1.1.2.1.9 NffvImage.setHeight Method

Sets image height.

Parameters

Parameters	Description
int height	Image height in pixels.

Java

```
public void setHeight(int height);
```

4.3.1.1.2.1.10 NffvImage.setHorizontalResolution Method

Sets the horizontal resolution of the image.

Parameters

Parameters	Description
float horizontalResolution	Horizontal resolution of image to set.

Java

```
public void setHorizontalResolution(float horizontalResolution);
```

4.3.1.1.2.1.11 NffvImage.setImageData Method

Creates an image from a byte array.

Parameters

Parameters	Description
byte [] imageData	Byte array that contains image data.

Java

```
public void setImageData(byte [] imageData);
```

4.3.1.1.2.1.12 NffvImage.setStride Method

Sets the stride (size of one row) of the image.

Parameters

Parameters	Description
int stride	Image stride.

Java

```
public void setStride(int stride);
```

4.3.1.1.2.1.13 NffvImage.setVerticalResolution Method

Sets the vertical resolution of the image.

Parameters

Parameters	Description
float verticalResolution	Vertical resolution of image to set.

Java

```
public void setVerticalResolution(float verticalResolution);
```

4.3.1.1.2.1.14 NffvImage.setWidth Method

Sets image width.

Parameters

Parameters	Description
int width	Image width in pixels.

Java

```
public void setWidth(int width);
```

4.3.1.1.3 NffvUser Class

Provides methods for working with users.

Class Hierarchy**Java**

```
public class NffvUser extends NativeObject;
```

NffvUser Methods

	Name	Description
	getID (🔗 see page 45)	Retrieves from a database user's ID. If a user was disposed or removed from engine an error is thrown.

	getNffvImage (see page 45)	Gets tan Image from the com.neurotechnology.Nffv (see page 36).
	toString (see page 45)	Gets a string representation of object.

Container

File: NffvUser.java

Package: com.neurotechnology.Nffv ([see page 36](#))

Module: Java Reference ([see page 36](#))

4.3.1.1.3.1 NffvUser Methods

4.3.1.1.3.1.1 NffvUser.getID Method

Retrieves from a database user's ID. If a user was disposed or removed from engine an error is thrown.

Returns

User's ID.

Java

```
public int getID() throws Exception;
```

4.3.1.1.3.1.2 NffvUser.getNffvImage Method

Gets tan Image from the com.neurotechnology.Nffv ([see page 36](#)).

Returns

NffvImage ([see page 41](#)) object.

Java

```
public NffvImage getNffvImage() throws Exception;
```

4.3.1.1.3.1.3 NffvUser.toString Method

Gets a string representation of object.

Returns

String representation of object.

Java

```
public String toString();
```

4.3.1.1.4 ScannerModule Class

Provides methods for setting and getting scanner names from the ScannerModule.

Class Hierarchy

Java

```
public class ScannerModule;
```

Methods

	Name	Description
	ScannerModule (see page 46)	Creates a new instance of the ScannerModule

ScannerModule Methods

	Name	Description
	getName (see page 46)	Gets a fingerprint scanner name.
	setName (see page 46)	Sets a name of a fingerprint scanner.

Container

File: ScannerModule.java

Package: com.neurotechnology.Nffv (🔗 see page 36)

Module: Java Reference (🔗 see page 36)

4.3.1.1.4.1 ScannerModule.ScannerModule Constructor

Creates a new instance of the ScannerModule

Parameters

Parameters	Description
String name	A name of a scanner to set.

Java

```
protected ScannerModule(String name);
```

4.3.1.1.4.2 ScannerModule Methods

4.3.1.1.4.2.1 ScannerModule.getName Method

Gets a fingerprint scanner name.

Returns

String that contains fingerprint scanner name.

Java

```
public String getName();
```

4.3.1.1.4.2.2 ScannerModule.setName Method

Sets a name of a fingerprint scanner.

Parameters

Parameters	Description
String name	A name of a scanner to set.

Java

```
protected void setName(String name);
```

4.3.1.2 Structs, Records, Enums

The following table lists structs, records, enums in this documentation.

Enumerations

	Name	Description
📄	NffvStatus (🔗 see page 46)	Enumerates NffvStatus values.

4.3.1.2.1 com.neurotechnology.Nffv.NffvStatus Enumeration

Enumerates NffvStatus values.

Java

```
public enum NffvStatus {  
};
```

Container

File: NffvStatus.java

Package: com.neurotechnology.Nffv ([see page 36](#))

Module: Java Reference ([see page 36](#))

5 Distribution Content

The Free Fingerprint Verification SDK contains these files and folders:

/bin

/Win64_x64 - this directory contains files for Windows-based operating systems running on 64-bit x86 CPU. This directory contains these files:

File	Description
CppSample.exe	An executable C++ sample application.
CSharpSample.exe	An executable C# sample application.
Neurotec.Biometrics.Nffv.dll	Provides the main functionality of the FFV SDK. Required Dll is Nffv.dll
Nffv.dll	Provides the main functionality of the FFV SDK.
VBNETSample.exe	An executable VB.NET sample application.
NffvJavaNative.dll	Files for Java sample.
NffvSample.jar	
NffvServer.exe	Nffv.dll helper.

/Win64_x64/FScanners - a directory containing scanner files:

Recommendations:

1. "FScanners" folder must be located in the same folder as Nffv.dll; "FScanners" folder must contain required scanners modules.
2. Unused scanner DLL files can be deleted. For example, if CrossMatch is not used, FScanners/NdmCrossMatch folder can be deleted.

/Documentation

File	Description
Free Fingerprint Verification SDK.pdf	Documentation file in pdf
Free Fingerprint Verification SDK.chm	Documentation file in chm
license.html	License file

/Include/Nffv - this directory contains header files.

/Lib /Win64_x64 - this directory contains lib files that are used by the Free Fingerprint Verification SDK.

/Samples/FreeFingerprintVerification - this directory contains files for sample applications. Folders under this directory:

Folder	Description
C++	This folder contains files for C++ sample application.
C#	This folder contains files for C# sample application.
Java	This folder contains files for Java sample application.
VB	This folder contains files for VB.NET sample application.

6 Error Codes

In this chapter the possible error codes are listed:

Error code	Error	Description
0	N_OK	No error.
-1	N_E_FAILED	Operation failed.
-2	N_E_CORE	Standard error has occurred (for internal use).
-3	N_E_NULL_REFERENCE	Null reference has occurred (for internal use).
-4	N_E_OUT_OF_MEMORY	There were not enough memory.
-5	N_E_NOT_IMPLEMENTED	Functionality is not implemented.
-6	N_E_NOT_SUPPORTED	Functionality is not supported.
-7	N_E_INVALID_OPERATION	Attempted to perform invalid operation.
-8	N_E_OVERFLOW	Arithmetic overflow has occurred.
-9	N_E_INDEX_OUT_OF_RANGE	Index is out of range (for internal use).
-10	N_E_ARGUMENT	Argument is invalid.
-11	N_E_ARGUMENT_NULL	Argument value is NULL where non-NULL value was expected.
-12	N_E_ARGUMENT_OUT_OF_RANGE	Argument value is out of range.
-13	N_E_FORMAT	Format of argument value is invalid.
-14	N_E_IO	Input/output error has occurred.
-15	N_E_END_OF_STREAM	Attempted to read file or buffer after its end.
-16	N_E_INVALID_ENUM_ARGUMENT	Argument is an invalid enum value.
-17	N_E_ARITHMETIC	Arithmetic error occurred.
-18	N_E_INVALID_CAST	The cast is invalid.
-19	N_E_DIRECTORY_NOT_FOUND	Directory not found.
-20	N_E_DRIVE_NOT_FOUND	Drive not found.
-21	N_E_FILE_NOT_FOUND	File not found.
-22	N_E_FILE_LOAD	Error loading module from file.
-23	N_E_PATH_TOO_LONG	Path is too long.
-24	N_E_SECURITY	Security error occurred.
-25	N_E_ABANDONED_MUTEX	The wait completed due to an abandoned mutex.
-26	N_E_BAD_IMAGE_FORMAT	Module has invalid format.
-27	N_E_DLL_NOT_FOUND	Module was not found.
-28	N_E_ENTRY_POINT_NOT_FOUND	Entry point not found in a module.
-29	N_E_FILE_FORMAT	File or stream format is invalid.
-30	N_E_TIMEOUT	Operation timed out.
-31	N_E_SOCKET	Socket error occurred.
-32	N_E_KEY_NOT_FOUND	Key not found.
-33	N_E_AGGREGATE	One or more errors occurred.
-34	N_E_OPERATION_CANCELED	Operation canceled.

-90	N_E_EXTERNAL	Error in external code has occurred (for internal use).
-91	N_E_WIN32	Win32 error has occurred.
-92	N_E_COM	COM error has occurred.
-93	N_E_CLR	CLR exception has occurred.
-94	N_E_SYS	Sys error occurred.
-95	N_E_MAC	macOS error occurred.
-96	N_E_CPP	C++ error occurred.
-97	N_E_JVM	JVM error occurred.
-200	N_E_NOT_ACTIVATED	Operation is not activated.

7 FAQ

There are listed frequently asked questions (FAQ) and answers to them.

1. When I have enrolled 10 users to a database and try to add more users I get an error message "Nffv already contains NF_ENGINE_MAX_USER_COUNT users. Code: -7". How can I add more users?

The Free Fingerprint Verification SDK allows to add up to 10 users to a database. If you need to add new users, you should remove other users.

If you have an intention of developing a larger scale application or system with unlimited users count, server or cluster support, please contact Neurotechnology for guidelines for other products.

2. Is it possible to save an original fingerprint image to a hard disk?

No, it is not possible. If you need to save an original fingerprint image, you should buy the VeriFinger SDK.

3. Can I use several different fingerprint scanners?

Yes, but you can only use one at a time.

4. What is a maximum matching score of two fingerprints?

There is no maximum score.

Index

▪

.NET Reference 26

A

API Reference 12
 About Neurotechnology 3
 About This Guide 2
 Additional Resources 2

C

C/C++ Reference 12
 Copyright Notice 1
 Distribution Content 48
 Enrollment 5
 Error Codes 49
 FAQ 51
 Feedback 1
 Fingerprint Scanners 4
 Fingerprints 5
 Free SDK vs. VeriFinger SDK 3
 How the Guide Is Organized 2
 How to Use Fingerprint Scanner 7
 Introduction 2
 Java Reference 36
 Matching Threshold 6
 NFFV_MAX_USER_COUNT 26
 NFFV_MAX_USER_COUNT macro 26
 Neurotec.Biometrics 27
 Neurotec.Biometrics namespace 27
 Classes 27
 Structs, Records, Enums 35
 Neurotec.Biometrics.Nffv 27
 Neurotec.Biometrics.Nffv.Cancel 32
 Neurotec.Biometrics.Nffv.Dispose 32
 Neurotec.Biometrics.Nffv.DllName 31
 Neurotec.Biometrics.Nffv.Enroll 32
 Neurotec.Biometrics.Nffv.GetAvailableScannerModules 33
 Neurotec.Biometrics.Nffv.GetUserById 33
 Neurotec.Biometrics.Nffv.MatchingThreshold 34

Neurotec.Biometrics.Nffv.MaxUserCount 31
 Neurotec.Biometrics.Nffv.Nffv 28, 29
 Neurotec.Biometrics.Nffv.QualityThreshold 34
 Neurotec.Biometrics.Nffv.UserCollection 29
 Neurotec.Biometrics.Nffv.UserCollection.Add 30
 Neurotec.Biometrics.Nffv.UserCollection.Contains 30
 Neurotec.Biometrics.Nffv.UserCollection.IndexOf 31
 Neurotec.Biometrics.Nffv.Verify 33
 Neurotec.Biometrics.NffvStatus 35
 Neurotec.Biometrics.NffvStatus enumeration 35
 Neurotec.Biometrics.NffvUser 34
 Neurotec.Biometrics.NffvUser.GetBitmap 35
 Neurotec.Biometrics.NffvUser.GetHBitmap 35
 Nffv Library 12
 Nffv Unit 12
 Functions 13
 Macros 26
 Structs, Records, Enums 25
 Nffv class 27, 37
 Cancel 32
 Dispose 32
 DllName 31
 Enroll 32
 GetAvailableScannerModules 33
 GetUserById 33
 MatchingThreshold 34
 MaxUserCount 31
 Nffv 28, 29, 37
 QualityThreshold 34
 Verify 33
 about Nffv class 27, 37
 cancel 38
 clearUsers 38
 contains 38
 enroll 38
 finalize 38
 getAvailableScannerModules 39
 getEngineStatus 39
 getMatchingThreshold 39
 getMaxUserCount 39
 getQualityThreshold 39
 getUserById 39

getUsers 40	NffvGetUserCount function 20
removeUser 40	NffvGetUserIndexById 20
removeUserID 40	NffvGetUserIndexById function 20
setEngineStatus 40	NffvImage class 41
setIntEngineStatus 40	about NffvImage class 41
setMatchingThreshold 41	getBufferedImage 42
setQualityThreshold 41	getHeight 42
verify 41	getHorizontalResolution 42
Nffv.UserCollection class 29	getImageData 42
Add 30	getImageIcon 43
Contains 30	getStride 43
IndexOf 31	getVerticalResolution 43
about Nffv.UserCollection class 29	getWidth 43
NffvCancel 14	setHeight 43
NffvCancel function 14	setHorizontalResolution 43
NffvClearUsers 15	setImageData 44
NffvClearUsers function 15	setStride 44
NffvEnroll 15	setVerticalResolution 44
NffvEnroll function 15	setWidth 44
NffvFreeMemory 16	NffvInitializeA 21
NffvFreeMemory function 16	NffvInitializeA function 21
NffvGetAvailableScannerModulesA 16	NffvInitializeW 21
NffvGetAvailableScannerModulesA function 16	NffvInitializeW function 21
NffvGetAvailableScannerModulesW 16	NffvRemoveUser 22
NffvGetAvailableScannerModulesW function 16	NffvRemoveUser function 22
NffvGetErrorMessageA 17	NffvSetMatchingThreshold 22
NffvGetErrorMessageA function 17	NffvSetMatchingThreshold function 22
NffvGetErrorMessageW 17	NffvSetQualityThreshold 23
NffvGetErrorMessageW function 17	NffvSetQualityThreshold function 23
NffvGetInfoA 17	NffvStatus 25
NffvGetInfoA function 17	NffvStatus Enumeration 25
NffvGetInfoW 18	NffvStatus_ 25
NffvGetInfoW function 18	NffvStatus_ enumeration 25
NffvGetMatchingThreshold 18	NffvUninitialize 23
NffvGetMatchingThreshold function 18	NffvUninitialize function 23
NffvGetQualityThreshold 19	NffvUser class 34, 44
NffvGetQualityThreshold function 19	GetBitmap 35
NffvGetUser 19	GetHBitmap 35
NffvGetUser function 19	about NffvUser class 34, 44
NffvGetUserById 19	getID 45
NffvGetUserById function 19	getNffvImage 45
NffvGetUserCount 20	toString 45

- NffvUserGetHBitmap 23
 - NffvUserGetHBitmap function 23
 - NffvUserGetId 24
 - NffvUserGetId function 24
 - NffvVerify 24
 - NffvVerify function 24
 - NoScanner enumeration member 35
 - Online Resources 3
 - Preface 1
 - Quality Threshold 6
 - QualityCheckFailed enumeration member 35
 - Questions 1
 - Quick Start 5
 - ScannerModule class 45
 - ScannerModule 46
 - about ScannerModule class 45
 - getName 46
 - setName 46
 - ScannerTimeout enumeration member 35
 - System Requirements 4
 - Target Audience 2
 - TemplateCreated enumeration member 35
 - Terminology 5
 - UserCanceled enumeration member 35
 - Using Sample Applications 7
 - Verification 6
 - com.neurotechnology.Nffv 36
 - com.neurotechnology.Nffv package 36, 46
 - Classes 36
 - com.neurotechnology.Nffv.Nffv 37
 - com.neurotechnology.Nffv.Nffv.Nffv 37
 - com.neurotechnology.Nffv.Nffv.cancel 38
 - com.neurotechnology.Nffv.Nffv.clearUsers 38
 - com.neurotechnology.Nffv.Nffv.contains 38
 - com.neurotechnology.Nffv.Nffv.enroll 38
 - com.neurotechnology.Nffv.Nffv.finalize 38
 - com.neurotechnology.Nffv.Nffv.getAvailableScannerModules 39
 - com.neurotechnology.Nffv.Nffv.getEngineStatus 39
 - com.neurotechnology.Nffv.Nffv.getMatchingThreshold 39
 - com.neurotechnology.Nffv.Nffv.getMaxUserCount 39
 - com.neurotechnology.Nffv.Nffv.getQualityThreshold 39
 - com.neurotechnology.Nffv.Nffv.getUserByID 39
 - com.neurotechnology.Nffv.Nffv.getUsers 40
 - com.neurotechnology.Nffv.Nffv.removeUser 40
 - com.neurotechnology.Nffv.Nffv.removeUserID 40
 - com.neurotechnology.Nffv.Nffv.setEngineStatus 40
 - com.neurotechnology.Nffv.Nffv.setIntEngineStatus 40
 - com.neurotechnology.Nffv.Nffv.setMatchingThreshold 41
 - com.neurotechnology.Nffv.Nffv.setQualityThreshold 41
 - com.neurotechnology.Nffv.Nffv.verify 41
 - com.neurotechnology.Nffv.NffvImage 41
 - com.neurotechnology.Nffv.NffvImage.getBufferedImage 42
 - com.neurotechnology.Nffv.NffvImage.getHeight 42
 - com.neurotechnology.Nffv.NffvImage.getHorizontalResolution 42
 - com.neurotechnology.Nffv.NffvImage.getImageData 42
 - com.neurotechnology.Nffv.NffvImage.getImageIcon 43
 - com.neurotechnology.Nffv.NffvImage.getStride 43
 - com.neurotechnology.Nffv.NffvImage.getVerticalResolution 43
 - com.neurotechnology.Nffv.NffvImage.getWidth 43
 - com.neurotechnology.Nffv.NffvImage.setHeight 43
 - com.neurotechnology.Nffv.NffvImage.setHorizontalResolution 43
 - com.neurotechnology.Nffv.NffvImage.setImageData 44
 - com.neurotechnology.Nffv.NffvImage.setStride 44
 - com.neurotechnology.Nffv.NffvImage.setVerticalResolution 44
 - com.neurotechnology.Nffv.NffvImage.setWidth 44
 - com.neurotechnology.Nffv.NffvStatus 46
 - com.neurotechnology.Nffv.NffvStatus enumeration 46
 - com.neurotechnology.Nffv.NffvUser 44
 - com.neurotechnology.Nffv.NffvUser.getId 45
 - com.neurotechnology.Nffv.NffvUser.getNffvImage 45
 - com.neurotechnology.Nffv.NffvUser.toString 45
 - com.neurotechnology.Nffv.ScannerModule 45
 - com.neurotechnology.Nffv.ScannerModule.ScannerModule 46
 - com.neurotechnology.Nffv.ScannerModule.getName 46
 - com.neurotechnology.Nffv.ScannerModule.setName 46
- ## N
- nfesNoScanner enumeration member 25
 - nfesQualityCheckFailed enumeration member 25
 - nfesScannerTimeout enumeration member 25
 - nfesTemplateCreated enumeration member 25
 - nfesUserCanceled enumeration member 25